DP优化之单调队列优化

DP是一种非常优秀的算法思想，它通过“重叠子问题、最优子结构、无后效性”实现高效的算法。

DP的效率取决于三个方面:

- **状态总数**;
- **每个状态的决策数**;
- **状态转移的计算量。**

这三个方面都可以进行优化。

- **(1)状态总数上的优化。**相当于搜索的各种剪枝，去除无效状态。降维，设计DP状态时尽量用低维的DP。
- **(2)减少决策数量**，即状态转移方程的优化。例如四边形不等式优化、斜率优化。
- **(3)状态转移计算量的优化。**例如:用预处理减少递推时间;用Hash表、单调队列、线段树减少枚举时间等。

1. 单调队列优化

单调队列是常见的DP优化技术，先回顾单调队列的概念。

1.1 单调队列的特征:

(1) 单调队列的实现。用双端队列实现，队头和队尾都能插入和弹出。手写双端队列很简单。

(2) 单调队列的单调性。队列内的元素具有单调性，从小到大，或者从大到小。

(3) 单调队列的维护。每个新元素都能进入队列，它从队尾进入队列时，为维护队列的单调性，应该与队尾比较，把破坏单调性的队尾弹出。

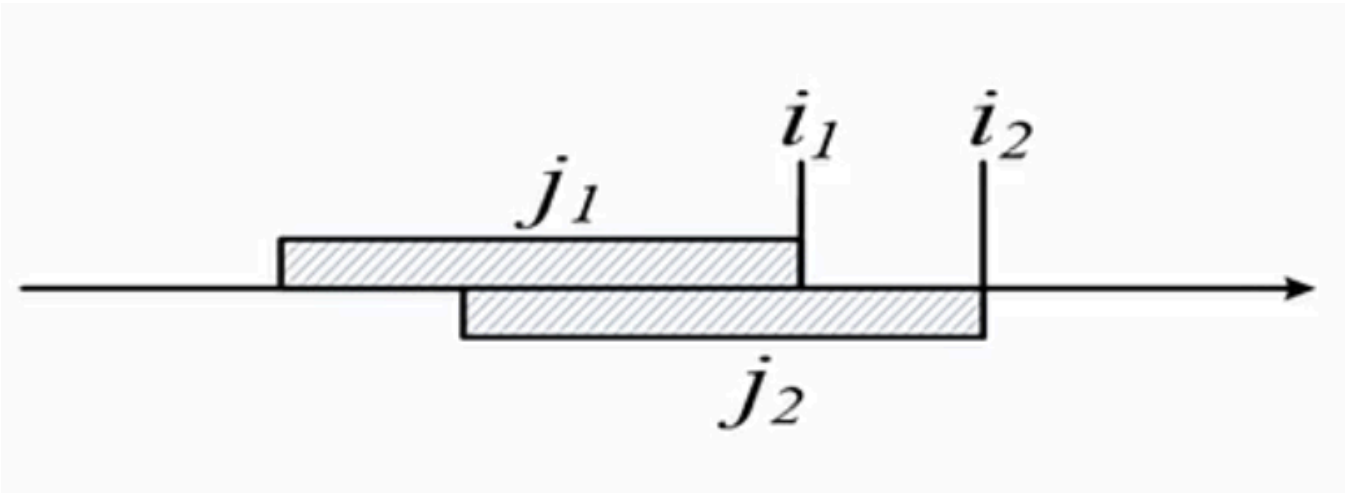
1.2 单调队列的基本应用

- 单调队列在DP优化中的基本应用，是对这样一类DP方程进行优化:

$$\mathrm{dp}[i] = \min\{\mathrm{dp}[j] + a[i] + b[j]\} \quad \mathrm{L}(i) \le j \le \mathrm{R}(i)$$

- 方程的特点是其中关于 i 的项 $a[i]$ 和关于 j 的项 $b[j]$ 是独立的，j 被限制在窗口 $[\mathrm{L}(i), \mathrm{R}(i)]$ 内。常见的如给定一个窗口值k，$i-k \leq j \leq i$
- 如果简单地对 i 做外层循环，对 j 做内层循环，复杂度为 $O(n^2)$。如果用单调队列优化，复杂度可提高到$O(n)$。

1.3 为什么单调队列可以优化



- 单调队列优化：把内外i、j两层循环，精简到一层循环。
- 原因：“外层i变化时，不同的i所对应的内层j的窗口有重叠”。
  如果把所有重叠的部分都优化掉，那么所有的j加起来只从头到尾遍历了一次，此时j的遍历实际上就是i的遍历了。

在窗口内处理的这些决策，有以下两种情况。

- **被排除的不合格决策**。内层循环j排除的不合格决策，在外层循环i增大时，需要重复排除。
- **未被排除的决策**。内层循环j未排除的决策，在外层循环i增大时，仍然能按原来的顺序被用到。

那么可以用单调队列统一处理这些决策，从而精简到只用一个循环，得到优化。

2. 单调队列优化例题

2.1 洛谷P2627 Mowing the Lawn

题目描述:

有一个包括n个正整数的序列，第i个整数是E；给定一个整数k，找这样的子序列，子序列中的数在原序列连续地不能超过k个。对子序列求和，问所有子序列中最大的和是多少。

例如，$n=5$，$\{7,2,3,4,5\}$，$k=2$,子序列$\{7$，2，4，$5\}$有最大和18，其中的连续部分是$\{7,2\}$、$\{4,5\}$，长度都不超过$k=2$。

思路:

定义 $dp[i]$为前i个整数的最大子序列和，状态转移方程:

$$dp[i]=max\{dp[j-1]+sum[i]-sum[j]\},i-k\le j\le i$$

其中$sum[i]$是前缀和，从E_1加到E_i。

把i看成定值，上述方程等价于下面的方程:

$$dp[i]=max\{dp[j-1]-sum[j]\}+sum[i],i-k\le j\le i$$

求$dp[i]$，就是找到一个决策j，$i-k\le j\le i$，使得$dp[j-1]-sum[j]$最大。对这个方程编程求解，如果简单地做i、j的循环，复杂度是$O(nk)$的，约等于$O(n^2)$.

- 对一个固定的i，用一个递减的单调队列求最大的$dp[j-1]-sum[j]$.记$ds[j]=dp[j-1]-sum[j]$,记这个i对应的最大值为 $dsmax[i]=max\{ds[j]\}$
- 用单调队列求$dsmax[i]$的步骤:
  - (1) $j=1$，$ds[1]$进队列。此时窗口内的最大值$dsmax[i]=ds[1]$。
  - (2) $j=2$，$ds[2]$进队列，讨论两种情况:
    - a. 若$ds[2]\ge ds[1]$，说明$ds[2]$更优，弹走$ds[1]$，$ds[2]$进队成为新队头，更新$dsmax[i]=ds[2]$。这一步排除了不好的决策，留下更好的决策。
    - b. 若$ds[2]<ds[1]$，$ds[2]$进队列。队头仍然是$ds[1]$，保持$dsmax[i]=ds[1]$。
    - 这两种情况下$ds[2]$都进队，是因为$ds[2]$比$ds[1]$更晚于离开窗口范围k，即存活时间更长

- (3) 继续以上操作，让窗口内的每个$j(i-k \le j \le i)$都有机会进队，并保持队列是从大到小的单调队列。

经过以上步骤，求得了固定一个i时的最大值$dsmax[i]$。

当i变化时，统一用一个单调队列处理，因为一个较小的i_1所排除的$ds[j]$，在处理后面较大的i_2时，也会被排除，没有必要再重新排除一次；而且较小的i_1所得到的队列，后面较大的i_2也仍然有用。

这样，每个$ds[j](1 \le j \le n)$都有机会进入队列一次，并且只进入队列一次，总复杂度$O(n)$。

参考代码

```
#include <bits/stdc++.h>
using namespace std;
const int N=100005;
long long n,k,e[N],sum[N],dp[N];
long long ds[N];          //ds[j] = dp[j-1]-sum[j]
int q[N],head=0,tail=1;   //递减的单调队列，队头最大
long long que_max(int j){
    ds[j] = dp[j-1]-sum[j];
    while(head<=tail && ds[q[tail]]<ds[j])  tail--;    //去掉不合格的队尾
    q[++tail]=j;                                //j进队尾
    while(head<=tail && q[head]<j-k)   head++;        //去掉超过窗口k的队头
    return ds[q[head]];                     //返回队头，即最大的dp[j-1]-sum[j]
}
int main(){
    cin >> n >> k;   sum[0] = 0;
    for(int i=1;i<=n;i++){
        cin >> e[i];  sum[i] = sum[i-1] + e[i];   //计算前缀和
    }
    for(int i=1;i<=n;i++)   dp[i] = que_max(i) + sum[i];  //状态转移方程
    cout << dp[n];
}
```

2.2 多重背包

多重背包问题：给定n种物品和一个背包，第i种物品的体积为c_i，价值为w_i，并且有m_i个，背

包的总容量为C。如何选择装入背包的物品，使装入背包的物品的总价值最大?

回顾用滚动数组实现的多重背包代码。

```
for(int i=1;i<=n;i++) //枚举每个物品
  for(int j=C;j>=c[i];j--)    //枚举背包容量
    for(int k=1;k<=m[i] && k*c[i]<=j;k++)
      dp[j]=max(dp[j],dp[j-k*c[i]] + k*w[i]);
```

状态转移方程为

$$dp[j] = max\{dp[j-k*c[i]] + k*w[i]\}, 1 \le k \le min\{m_i, j/c_i\}$$

代码为i、j、k三重循环。其中，循环i、j互相独立，没有关系，不能优化；循环j、k是相关的，k在j上有滑动窗口、所以目标是优化j、k这两层循环，此时可以把与i有关的部分看作定值。

对比单调队列的标准转移方程，

$$\mathrm{dp}[i] = \min\{\mathrm{dp}[j] + a[i] + b[j]\} \quad \mathrm{L}(i) \le j \le \mathrm{R}(i)$$

相差太大，似乎并不能应用单调队列。

回顾单调队列优化的实质，外层i变化时，不同的i所对应的内层j的窗口有重叠。状态转移方程$dp[j] = max\{dp[j-k*c[i]] + k*w[i]\}$的外层循环是$j$，内层循环是$k$，$k$的滑动窗口是否重叠？通过观察$j$-k$c_i$的变化情况，首先对比外层$j$和$j$+1，让$k$从1递增，观察它们的$j$-k$c_i$

j：　$j-3c_i$　　$j-2c_i$　　$j-c_i$

$j+1$：　$j+1-3c_i$　　$j+1-2c_i$　　$j+1-c_i$

可以发现没有发生重叠。但是如果对比j和j+c_i，发生了重叠。

j：　$j-3c_i$　　$j-2c_i$　　$j-c_i$

$j+c_i:\quad j+c_i-4c_i \qquad j+c_i-3c_i \qquad j+c_i-2c_i$

这样我们推理出，当$j=j$，$j+c_i$，$j+2c_i$，...时有重叠；进一步推理出，当j除以c_i的余数相等时，这些j对应的内层k发生重叠。那么，如果把外层j的循环改为按j除以c_i的余数相等的值进行循环，就能利用单调队列优化了。

下面把原状态转移方程变换为可以应用单调队列的标准方程。原方程为

$dp[j]=max\{dp[j-k*c[i]]+k*w[i]\},1\le k\le min\{m_i,j/c_i\}$

令$j=b+yc_i$，其中$b=j\%c_i$，b为j除以c_i得到的余数；$y=j/c_i$，y为j整除c_i的结果。把j带入原方程，得

$dp[b+yc_i]=max\{dp[b+(y-k)c_i]+k*w_i\},1\le k\le min\{m_i,y\}$

令$x=y-k$，代入得

$dp[b+yc_i]=max\{dp[b+xc_i]-xv_i+y*w_i\},y-min\{m_i,y\}\le x\le y$

与标准方程$dp[i]=min\{dp[j]+a[i]+b[j]\}$对比，几乎一样。

用单调队列处理决策$dp[b+xc_i]-xv_i$，下面详见代码。

参考代码

```cpp
//洛谷 P1776: 单调队列优化多重背包
#include<bits/stdc++.h>
using namespace std;
const int N=100010;
int n,C;
int dp[N],q[N],num[N];
int w,c,m;                          //物品的价值w、体积c、数量m
int main(){
    cin >> n >> C;                  //物品数量n，背包容量W
    memset(dp,0,sizeof(dp));
    for(int i=1;i<=n;i++){
        cin>>w>>c>>m;                //物品i的价值w、体积c、数量m
        if(m>C/c) m = C/c;           //计算 min{m, j/c}
        for(int b=0;b<c;b++){        //按余数b进行循环
            int head=1, tail=1;
            for(int y=0;y<=(C-b)/c;y++){      //y = j/c
                int tmp = dp[b+y*c]-y*w;      //用队列处理tmp = dp[b + xc] - xw
                while(head<tail && q[tail-1]<=tmp)   tail--;
                q[tail] = tmp;
                num[tail++] = y;
                while(head<tail && y-num[head]>m)    head++;
//约束条件y-min(mi,y)≤x≤y
                dp[b+y*c] = max(dp[b+y*c],q[head]+y*w);  //计算新的dp[]
            }
        }
    }
    cout << dp[C] << endl;
    return 0;
}

```